

Math 443 Homework 1

Mark Stanley

January 28, 2025

Exercise 1

Solve the following system of linear equations of size 8×8 using LU factor-ization:

$$Ax = b,$$

where A is the coefficient matrix and b is the right-hand side vector. Use the given A and b below:

$$A = \begin{bmatrix} 4 & -2 & 1 & 0 & 0 & 0 & 0 & 0 \\ -2 & 4 & -2 & 1 & 0 & 0 & 0 & 0 \\ 1 & -2 & 4 & -2 & 1 & 0 & 0 & 0 \\ 0 & 1 & -2 & 4 & -2 & 1 & 0 & 0 \\ 0 & 0 & 1 & -2 & 4 & -2 & 1 & 0 \\ 0 & 0 & 0 & 1 & -2 & 4 & -2 & 1 \\ 0 & 0 & 0 & 0 & 1 & -2 & 4 & -2 \\ 0 & 0 & 0 & 0 & 0 & 1 & -2 & 4 \end{bmatrix}, b = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \\ 8 \end{bmatrix}$$

Proof. The solution to this problem is written in Python and the bonus question at the bottom. The entire file is uploaded with this file, as well as the relevant code areas associated with each subsection question. The solutions are pretty simple, the LU-factorization goes through the Gaussian Elimination code we viewed in class, expect tracks the elementary matrices used for row reduction, and then returns their product. The forward and backward substitutions differ slightly, as they use the dot product as opposed to the summation, but these statements are equivalent. $\square$

1

Rewrite A as LU , provide all the steps.

The code can be written as follows, and uses numpy arrays.

```

1 import numpy as np
2
3 def lu_factorization_np(A_orig: np.array) -> Optional[Tuple[np.array, np.
  array]]:
4     """
5     Performs LU factorization on a square matrix A. Specifically finds L
    and U through Gaussian elimination, which means that it isn't
    necessarily quicker on a single system of equations than Gaussian
    elimination.
6
7     :param A: a regular matrix (np array)
8     :return: a tuple containing the upper and lower triangular matrices L
    and U
9     that satisfy A = LU
10    """
11    # make copies so as not to mutate the original arrays
12    A = A_orig.copy()
13    n = len(A)
14
15    elementary_matrices = []
16    for j in range(n):
17        if A[j, j] == 0:
18            print("A is irregular")
19            return None
20
21        else:
22            for i in range(j + 1, n):
23                factor = A[i, j] / A[j, j]
24                A[i, j:] = np.subtract(A[i, j:], factor * A[j, j:])
25
26                elementary_matrix = np.identity(n)
27                elementary_matrix[i, j] = factor
28                elementary_matrices.append(elementary_matrix)
29
30    L = elementary_matrices.pop()
31    for matrix in reversed(elementary_matrices):
32        L = np.matmul(matrix, L)
33
34    return L, A

```

We will list all numbers in the matrices as floating point numbers, though the canonical answer should be in rational numbers.

$$L = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -0.5 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0.25 & 0.5 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0.33333333 & -0.5 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0.33333333 & -0.51428571 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.34285714 & -0.51315789 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0.34539474 & -0.51363636 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0.34545455 & -0.51415094 & 1 \end{bmatrix}$$

$$U = \begin{bmatrix} 4 & -2 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 3 & -1.5 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 3 & -1.5 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2.91666667 & -1.5 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 2.89523810 & -1.48571429 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 2.89473684e & -1.48684211e & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 2.89090909 & -1.48636364 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 2.89033019 \end{bmatrix}$$

The resulting matrix multiplication $L \times U$ has the greatest floating point difference of $7.67063181e^{-17}$ from the original A , and this only occurs on the last row of the matrix. This seems pretty good.

2

Forward Substitution: Solve $Ly = b$ for y .

```
1 def forward_substitution_np(L_orig: np.array, b_orig: np.array) ->
  Optional[np.array]:
2     """
3     Perform forward substitution on a matrix L and a vector b to solve the
4     system of equations Lc = b.
5
6     :param L_orig: the lower unitriangular square matrix L
7     :param b_orig: the system of equations you are solving for
8     :return: the solution vector c
9     """
10    L = L_orig.copy()
11    b = b_orig.copy()
12    n = len(L)
13
14    c = np.zeros(n)
15    for i in range(n):
16        c[i] = b[i] - np.dot(L[i, :i], c[:i])
17        # again we can use the dot product
18
19    return c
```

The only thing noteworthy in this code is the use of a dot product instead of the summation. This code gives us the resulting vector of.

$$y = \begin{bmatrix} 1 \\ 2.5 \\ 4 \\ 5.16666667 \\ 6.32380952 \\ 7.47368421 \\ 8.65454545 \\ 9.86792453 \end{bmatrix}$$

3

```
1 def back_substitution_np(U_orig: np.array, c_orig: np.array) -> Optional[
    np.array]:
2     """
3     Perform back substitution on a matrix U and a vector c to solve the
4     system of equations  $Ux = c$ .
5
6     :param U_orig: the upper triangular matrix U
7     :param c_orig: the vector you are solving for x
8     :return: the solution vector x
9     """
10    U = U_orig.copy()
11    c = c_orig.copy()
12    n = len(U)
13
14    x = np.zeros(n)
15    for i in reversed(range(n)):
16        x[i] = (c[i] - np.dot(U[i, i + 1:], x[i + 1:])) / U[i, i]
17        # we can use the dot product as opposed to the summation
18
19    return x
```

This yields

$$y = \begin{bmatrix} 0.375357 \\ 0.93512852 \\ 1.36882905 \\ 1.74785802 \\ 2.51529988 \\ 3.84169727 \\ 4.74908201 \\ 3.41411669 \end{bmatrix}$$

Which when multiplied Ax gives us exactly the starting vector

Bonus Exercise:

Given some regular square matrix $A = LU$, where L, U are its respective LU-factorized forms, prove that if $Lc = b$ and $Ux = c$, then x is the same as $Ax = b$.

Proof. This seems initially easy, but there is nothing intrinsic to why *any* matrix has an inverse. Here, we must utilize the fact that L is the product of elementary matrices, each of which has a defined inverse U_i , and vice versa for U . Thus, we know some left inverse exists (namely, $L = L_1 L_2 \dots L_n \implies L^{-1} = U_n \dots U_1$).

Thus, we now have $c = L^{-1}b$, which means that $Ux = L^{-1}b$, and therefore $LUx = b$, finally $Ax = b$

□